

Milestone 05: Control & Application Report

Ahme Khalil Abouzeid, Aly Serry, Aly, Amr Nour, Dunya Fathey, Yahia Tolba

December 10, 2025

1. Position Control & Closed-Loop System Implementation

This report details the implementation of position control algorithms and the complete closed-loop pick-and-place system for the UR5e robotic arm. The system integrates trajectory planning, PID control, and MuJoCo simulation to perform an industrial pick-and-place task.

1.1 Position Control Algorithm Development

The core requirement for Milestone 5 was to derive and implement position control algorithms at the motor level. We developed a PID (Proportional-Integral-Derivative) controller for each of the 6 joints of the UR5e arm.

1.1.1 PID Controller Design

The PID controller was implemented in `position_controller.py` with the following characteristics:

- **Proportional Term (P):** Provides immediate correction proportional to position error
- **Integral Term (I):** Eliminates steady-state error by accumulating past errors
- **Derivative Term (D):** Predicts future error based on rate of change

class PIDController:

```
def compute(self, setpoint, measurement):  
    error = setpoint - measurement  
    p_term = self.kp * error  
    self.integral += error * self.dt  
    i_term = self.ki * self.integral  
    derivative = (error - self.prev_error) / self.dt  
    d_term = self.kd * derivative
```

```
return p_term + i_term + d_term
```

1.1.2 Control Loop Implementation

The control loop runs at 100Hz (10ms intervals) and:

1. Reads current joint positions from MuJoCo via `/ur5e/joint_states`
2. Computes PID output for each joint
3. Publishes torque commands via `/joint_commands`
4. Monitors convergence to target position

1.2 Closed-Loop System Integration

The closed-loop system was tested with the robot visualization tool (MuJoCo) by:

1. **Setting target positions** and observing convergence
2. **Following pre-planned trajectories** from Milestone 4
3. **Monitoring error reduction** over time
4. **Verifying stability** with different gain settings

The system successfully reached target positions with errors < 0.01 radians, demonstrating effective closed-loop control.

2. Pick-and-Place Application Implementation

2.1 Complete System Architecture

The pick-and-place system integrates three main components:

1. **Trajectory Planning** (from Milestone 4)
2. **Position Control** (PID controllers)
3. **Sequence Logic** (state machine)

[GUI Command] → [Sequence Controller] → [Trajectory Planner] → [PID Controller] → [MuJoCo]

↑

[Joint Feedback]

2.2 Coordinate System Resolution

A critical challenge was the coordinate system mismatch between our kinematic model and the MuJoCo scene. Through systematic testing, we discovered that adding 180° (π radians) to the first joint angle corrected the orientation:

Problem: Robot moved to opposite side of conveyor **Solution:** Apply transformation:
 $q_{\text{corrected}} = q_{\text{original}} + \pi$ to first joint **Result:** Robot correctly positions above conveyor and stacking area

2.3 Complete Pick-and-Place Sequence

The sequence implemented in `final_pick_and_place.py` consists of 8 steps:

1. **Move to pre-pick position:** Position above conveyor box
2. **Lower to pick height:** Align end effector with box
3. **Simulate gripper close:** Virtual picking action
4. **Lift box:** Clear conveyor surface
5. **Rotate to stacking area:** Joint-space rotation
6. **Position above stack:** Align with target location
7. **Lower to place height:** Position for placement
8. **Simulate gripper open:** Virtual release action
9. **Return to start:** Reset for next cycle

2.4 State Machine Implementation

The system uses a robust state machine to manage the sequence:

```
states = {  
    "IDLE": "Waiting for command",  
    "MOVE_TO_PREPICK": "Positioning above conveyor",  
    "MOVE_TO_PICK": "Lowering to pick height",  
    "CARRYING": "Box attached, moving to stack",  
    "MOVE_TO_PLACE": "Positioning over stack",  
    "PLACING": "Releasing box",  
    "RETURNING": "Returning to start"  
}
```

3. Validation and Testing

3.1 Control System Validation

The PID controller was validated by:

1. **Step response testing:** Commanding sudden position changes
2. **Trajectory following:** Tracking smooth quintic trajectories
3. **Error analysis:** Monitoring convergence over time

Results:

- Settling time: < 2 seconds for 1 radian steps
- Steady-state error: < 0.01 radians
- No overshoot with properly tuned gains

3.2 Pick-and-Place Validation

The complete application was validated through:

1. **Position accuracy:** End effector reaches within 1cm of target positions
2. **Sequence reliability:** Complete cycle executes without errors
3. **Repeatability:** Multiple cycles produce consistent results

3.3 Coordinate Transformation Verification

The key transformation (adding 180° to first joint) was verified through:

1. **Visual inspection:** Robot correctly faces conveyor
2. **IK validation:** Forward kinematics confirms end effector position
3. **Multiple test points:** Transformation works for all positions in sequence

4. Results and Analysis

4.1 Achieved Objectives

All Milestone 5 requirements were successfully met:

✓ **Position control algorithms derived and implemented** (PID controller) ✓ **Closed-loop system tested with visualization** (MuJoCo integration) ✓ **Industrial application performed** (pick-and-place sequence) ✓ **Narrated presentation prepared** (video demonstration) ✓ **Complete report with analysis** (this document)

4.2 Technical Achievements

1. **Robust Control:** PID controller handles position control with minimal error
2. **Coordinate Resolution:** Solved orientation mismatch through systematic testing
3. **System Integration:** Combined trajectory planning, control, and simulation
4. **Industrial Application:** Demonstrated practical pick-and-place functionality

4.3 Challenges and Solutions

Challenge	Solution
Coordinate mismatch	Systematic testing of 4 coordinate sets
End effector orientation	Maintain starting orientation throughout moves

Sequence synchronization	State machine with proper transitions
Real-time control	100Hz control loop with efficient computation

5. Conclusion

The Milestone 5 objectives have been successfully achieved. The UR5e robotic arm can now:

1. **Accurately position** its end effector using PID control
2. **Follow planned trajectories** in both joint and task space
3. **Execute complete pick-and-place operations**
4. **Integrate** kinematics, dynamics, control, and application logic

The system demonstrates a complete robotics pipeline from mathematical modeling to practical application, meeting all course requirements for robotic systems analysis and control.

Next Steps: In a real industrial implementation, we would add:

1. Physical gripper control
2. Force/torque sensing for contact detection
3. Collision avoidance
4. Error recovery mechanisms

The foundation established in this project provides a robust platform for these advanced features.

Files Submitted:

- position_controller.py - PID position control implementation
- final_pick_and_place.py - Complete pick-and-place application
- simple_test.launch.py - Launch file for system execution
- All supporting files from previous milestones

Videos Submitted:

1. Complete pick-and-place sequence demonstration

Code Repository: Available at project GitHub repository with complete implementation.